CineComic-MML

Stage-by-Stage Build and Evaluation Strategy

A multimodal video-to-comic system: Whisper, diarization, CLIP, ArcFace, active speaker detection, emotion fusion, comic generation

Final-year project guide: what to build, which technology to use, how to connect each stage, how to evaluate it, and when to move on

How to read this guide. Section 1-4 explain the strategy and how stages connect. Section 5 is the detailed guide for Phases 0-13, each with the same layout (goal, tech, steps, output format, evaluation, tests, gate, connections, failures). Sections 6-12 cover datasets, the gate summary, ablations, risks, timeline, configs and code skeletons.

Note on thresholds. Numeric gates in this guide are suggested starting points, not published standards. Run the baseline first, then set and justify your own thresholds in the report. Model names are examples; check that each is available and what licence it carries before relying on it.

VS code Files

mkdir CineComic_MML

cd CineComic_MML

mkdir configs

mkdir data

mkdir data\raw

mkdir data\metadata

mkdir data\audio

mkdir data\shots

mkdir data\frames

mkdir data\subtitles

mkdir data\diarization

mkdir data\faces

```
mkdir data\tracks

mkdir data\embeddings

mkdir data\alignment

mkdir data\keyframes

mkdir data\emotions

mkdir data\layout

mkdir data\outputs


mkdir src

mkdir src\ingestion

mkdir src\audio

mkdir src\video

mkdir src\vision

mkdir src\speaker

mkdir src\alignment

mkdir src\keyframes

mkdir src\emotion

mkdir src\layout

mkdir src\comic

mkdir src\evaluation


mkdir tests

mkdir results
```

```
mkdir logs

mkdir notebooks

mkdir models

New-Item configs\phase1.yaml -ItemType File

New-Item src\ingestion\__init__.py -ItemType File

New-Item src\ingestion\video_ingestion.py -ItemType File

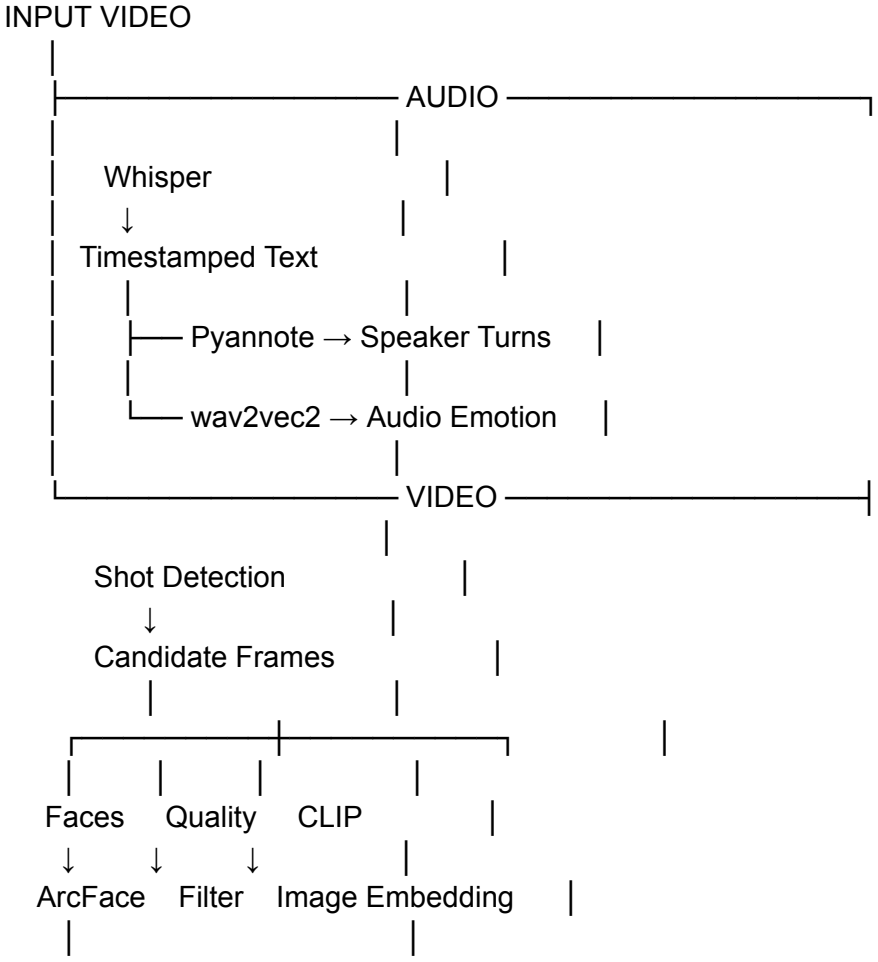
New-Item tests\test_phase1.py -ItemType File

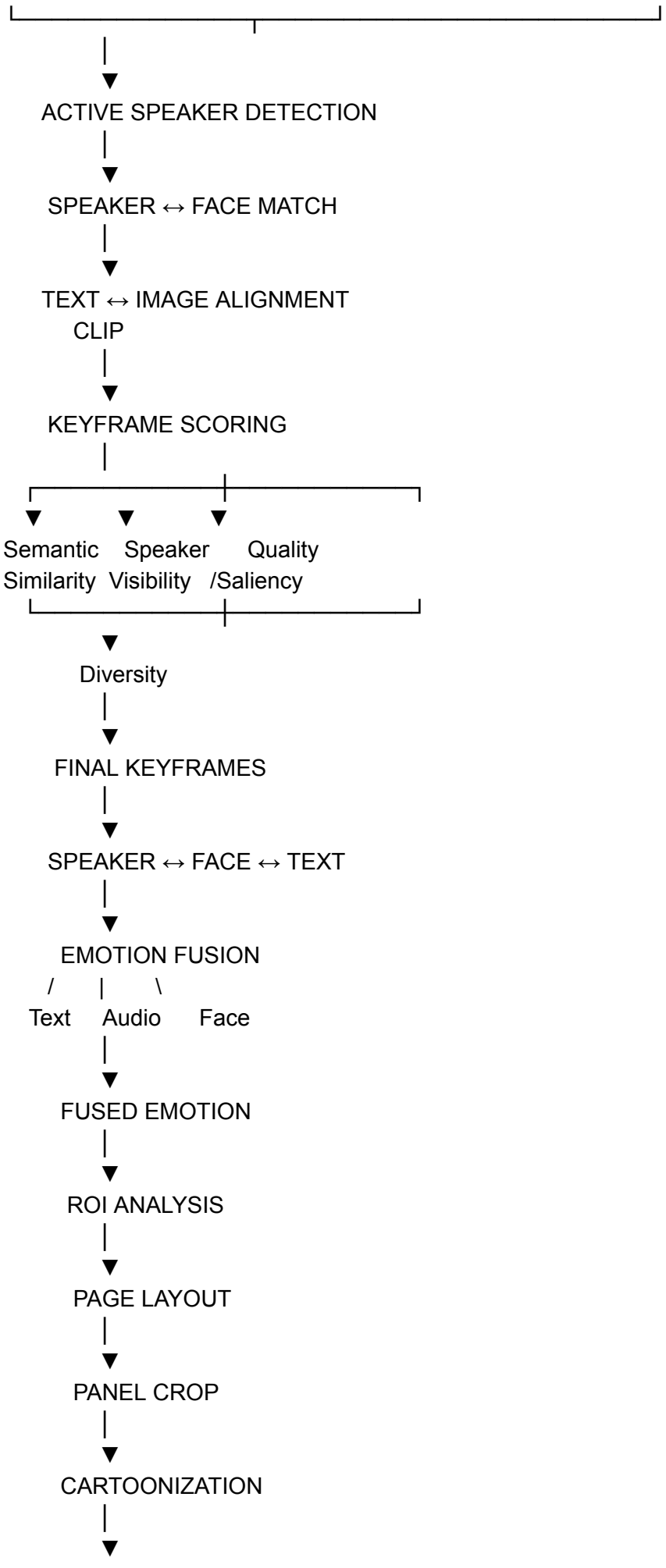
New-Item requirements.txt -ItemType File

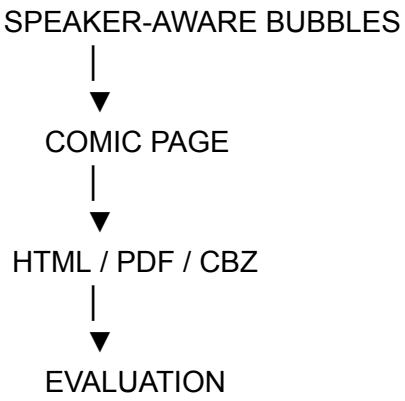
New-Item README.md -ItemType File

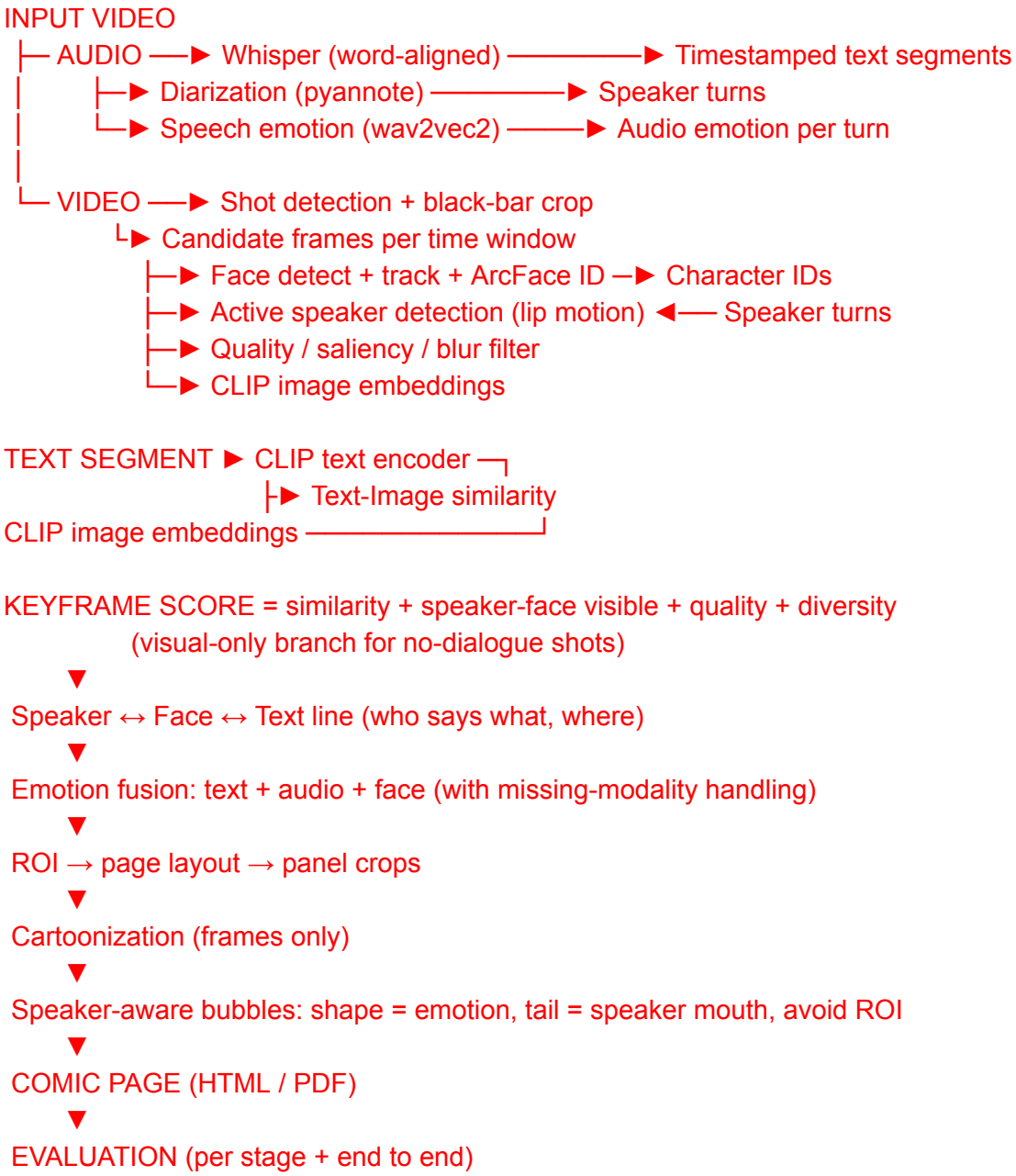
New-Item main.py -ItemType File
```

Structure




ACTIVE SPEAKER DETECTION
SPEAKER ↔ FACE MATCH
TEXT ↔ IMAGE ALIGNMENT
CLIP
KEYFRAME SCORING
Semantic Similarity
Speaker Visibility
Quality /Saliency
Diversity
FINAL KEYFRAMES
SPEAKER ↔ FACE ↔ TEXT
EMOTION FUSION
Text
Audio
Face
FUSED EMOTION
ROI ANALYSIS
PAGE LAYOUT
PANEL CROP
CARTOONIZATION


SPEAKER-AWARE BUBBLES
COMIC PAGE
HTML / PDF / CBZ
EVALUATION


INPUT VIDEO
AUDIO
Whisper (word-aligned)
Timestamped text segments
Diarization (pyannote)
Speaker turns
Speech emotion (wav2vec2)
Audio emotion per turn
VIDEO
Shot detection + black-bar crop
Candidate frames per time window
Face detect + track + ArcFace ID
Character IDs
Active speaker detection (lip motion)
Speaker turns
Quality / saliency / blur filter
CLIP image embeddings
TEXT SEGMENT
CLIP text encoder
Text-Image similarity
CLIP image embeddings
KEYFRAME SCORE = similarity + speaker-face visible + quality + diversity
(visual-only branch for no-dialogue shots)
Speaker ↔ Face ↔ Text line (who says what, where)
Emotion fusion: text + audio + face (with missing-modality handling)
ROI → page layout → panel crops
Cartoonization (frames only)
Speaker-aware bubbles: shape = emotion, tail = speaker mouth, avoid ROI
COMIC PAGE (HTML / PDF)
EVALUATION (per stage + end to end)

Contents

- 1. Strategy overview and the stage template
- 2. The CineComic baseline and what changes
- 3. Dependency graph and data contract between stages • 4.

Global conventions (time, IDs, coordinates, config, caching) • 5.

Phase-by-phase guide (Phases 0-13)

- 6. Datasets and annotation plan
- 7. Gate summary table
- 8. Baseline vs MML configs and ablation design
- 9. Evaluation metrics cheat-sheet
- 10. Risk register and what to cut
- 11. Sixteen-week timeline
- 12. Folder structure and code skeletons
- 13. Corrections to earlier drafts
- 14. Viva preparatio

1. Strategy Overview

The central rule: **never build the whole multimodal system at once.** If Whisper, pyannote, CLIP, ArcFace, emotion models and layout are wired together on day one and the comic looks wrong, nothing tells you which component failed. Instead, build one stage at a time. A stage only counts as finished when it has produced a saved artifact and passed a measured gate.

1.1 The seven-step stage template

1. **Input:** which earlier artifacts the stage reads.
2. **Processing:** the model or algorithm.
3. **Output:** one saved artifact in a documented JSON/NumPy format.
4. **Evaluation:** metrics computed against a small hand-labelled set.
5. **Gate:** an explicit pass/fail threshold.
6. **Checkpoint:** the artifact and metrics are frozen and logged.
7. **Next stage** starts only after the gate passes.

```
IMPLEMENT -> UNIT TEST -> RUN ON SMALL SAMPLE -> SAVE OUTPUT
 -> VISUALISE / INSPECT -> COMPUTE METRICS -> PASS?
 |-- NO -> FIX, repeat
 `-- YES -> save checkpoint -> next stage
```

1.2 Why this makes a stronger final-year project

- You can show stage-wise evaluation, not just a final comic that looks nice.
- Failures are traceable to a single stage, so debugging is fast.
- Each stage becomes a result table in your report (ASR table, diarization table, keyframe table, and so on).

• The baseline-versus-MML comparison becomes a reproducible experiment instead of an opinion. **1.3 The three multimodal ideas the project demonstrates**

- **Audio-text:** Whisper turns speech into text; diarization turns it into who-spoke-when.
- **Text-image:** CLIP aligns dialogue with frames in one shared embedding space (true joint embedding, which the original CineComic lacks).
- **Audio-visual:** active speaker detection links a voice to a face; emotion fusion combines text, voice and face.

2. The CineComic Baseline and What Changes

The original CineComic (github.com/reubendinny/Cinecomic) runs a fixed sequential pipeline, confirmed from its app.py: subtitles (Whisper) -> keyframes (GoogLeNet features + DSN + dialogue grouping) -> black-bar crop -> panel layout (ROI + templates) -> speech bubbles (text emotion + Dlib mouth position) -> page creation -> cartoonization -> HTML page. It is a late-fusion system: each modality has its own model and the results are joined by timestamps and layout rules.

Keyframe choice	DSN score + dialogue grouping, visual only	CLIP text-image similarity + speaker visibility + quality + saliency + diversity	6, 8
Speaker	Not attributed	Diarization + active speaker detection + Hungarian mapping	3, 7
Identity	None	Face tracking + ArcFace character IDs	5
Emotion	Text only	Text + audio + face with missing-modality masking	9
Bubble tail	Nearest detected mouth (Dlib)	Mouth of the matched speaker; off-screen and unknown handling	12
Cartoonization order	After bubbles in the script	After selection, before bubble overlay; analysis always on original frames	11, 12
Evaluation	None described in README	Stage metrics, ablations, user study, statistics	13
Output	HTML viewer	HTML + PDF + CBZ	12
Execution	Runs inside the web request	Staged artifacts, cached, resumable	0

Keep from the original: the modular folder-per-stage idea, the black-bar crop, pHash/Hamming duplicate removal, template-based layout and the HTML output template. **Replace or add:** everything in the right-hand column above.

3. Dependency Graph and Data Contract

This section answers 'how do the stages connect?'. Every arrow below is a file that one stage writes and another reads. If a stage's output format changes, every downstream reader breaks, so the formats are fixed early.

```
P0 Env ------------------------------------------------> (used by all)
P1 Ingestion --> wav, metadata
 |--> P2 Whisper --> transcript.json (segment_id, start, end, text)
 | |--> P3 Diarization --> segment_speakers (SPEAKER_xx)
 | |--> P6 CLIP text embeddings
 | `--> P9 Text emotion
 `--> P4 Shots + candidate frames (frame_id, t, shot_id)
 |--> P5 Faces/tracks/CHAR_ids --+
 `--> P6 CLIP image embeddings |
```

```
P3 + P5 --> P7 Active speaker + Speaker<->Face (SPEAKER_xx -> CHAR_yy)
P4 + P5 + P6 + P7 --> P8 Keyframe selection (keyframes.json)
P2 + P3 + P5 + P7 --> P9 Emotion fusion (text + audio + face)
P8 + P5 --> P10 ROI + layout (crop transforms)
P10 --> P11 Cartoonization
P2 + P7 + P9 + P10 + P11 --> P12 Bubbles + page + export
P12 --> P13 Evaluation vs baseline
```

Data contract: which fields each stage must write

P1	video_id, fps, duration, wav path	2, 3, 4, 9
P2	segment_id, start, end, text, words	3, 4, 6, 9, 12
P3	turns, segment -> speaker (SPEAKER_xx), overlap_ratio	7, 9, 12
P4	crop_box, shot_id, frame_id, t, sharpness, segment_candidates	5, 6, 8, 10
P5	track_id, character_id, face box, mouth point, embedding index	7, 8, 9, 10, 12
P6	clip_sim and z-score per (segment, frame); image embeddings	8
P7	segment -> speaker, character_id, track_id, conf, on_screen	8, 9, 12
P8	ordered panels: frame_id, segment_ids, character, score breakdown	9, 10
P9	fused emotion label + confidence per segment	12
P10	pages, panels, crop transform (offset, scale), safe zones	11, 12
P11	styled panel images	12
P12	page.json, HTML, PDF, CBZ	13

Rule: a stage may read only artifacts, never another stage's in-memory objects. That is what allows you to re-run one stage in isolation, swap a model, and compare baseline against MML.

4. Global Conventions

4.1 Time

All times are float seconds from the start of the constant-frame-rate video. Frame index = round(t * fps). Never mix frame numbers and seconds in saved files.

4.2 Identifiers

video_id	clip_001 or SHA-256 prefix	P1	One input video
segment_id	1, 2, 3 ... in time order	P2	One dialogue line (primary key across modalities)
shot_id	integer	P4	One camera shot
frame_id	s02_f003	P4	One candidate frame
speaker	SPEAKER_00 ...	P3	Anonymous voice
track_id	integer, unique per video	P5	A face followed through one shot
character_id	CHAR_01 ...	P5	Person across the whole video

4.3 Coordinates

Faces and mouths are detected on **original-frame pixels**. Every later transform is stored: black-bar crop offset, panel crop offset, scale to panel size, panel position on page. The balloon tail is computed by applying this chain to the mouth point. Store (x, y, w, h) boxes as pixel values in the original frame and say so in the schema.

4.4 Configuration and reproducibility

- All thresholds, model names and weights live in configs/*.yaml; none are hard-coded.
- Set random seeds (Python, NumPy, PyTorch) from the config.
- Save each stage's metrics JSON next to its artifact, including the config hash and library versions.
- Cache by (video_id, stage, config-hash) so unchanged stages are skipped.
- Log with Python logging to logs/{video_id}.log; never print only to the screen.

4.5 Splits

Divide your clips into a **development** set (used to tune thresholds and weights), a **validation** set and a **test** set (touched once, in Phase 13). Reporting numbers on clips that you tuned on is the most common way to overstate results.

5. Phase-by-Phase Guide

Each phase below uses the same layout. Work through them in order and do not start a phase until the previous gate has passed. Phases 2-3 (audio) and 4-5 (video) are independent branches that you could build in parallel with a teammate, but they must both pass before Phase 6-7.

Phase 0: Environment and Project Foundation

Create a reproducible project skeleton before touching any model. Every later phase reuses the config loader, logger, artifact-path helpers and stage base class built here, so a mistake now is paid for 13 times later.

Input	Empty repository, a machine (CPU or GPU).
Tech stack	Python 3.10/3.11, venv or conda, PyTorch, OpenCV, FFmpeg, PyYAML/OmegaConf, pytest, Git (DVC optional), Python logging.
Output artifacts	environment_report.json, configs/*.yaml, folder skeleton, BaseStage class.
Feeds into	All phases.

Processing steps

1. Create the folder skeleton (data/, src/, configs/, tests/, results/, logs/) exactly as in Section 12.
2. Pin every dependency version in requirements.txt (heavy models may need separate environments; see pitfalls).
3. Write env_check.py: import every library, print versions, detect CUDA, run `ffmpeg -version`. 4. Write the shared utilities: load_config(), set_seed(), get_logger(), artifact_path(video_id, stage). 5. Write BaseStage with three methods: run(video_id), evaluate(video_id), save(). Every phase subclasses it. 6. Add caching: if the artifact for (video_id, stage, config-hash) already exists, skip the run.

Output format (example)

```
{ "python": "3.11.9", "torch": "2.x", "cuda": false,
 "opencv": "4.9.0", "ffmpeg": "6.x", "whisper": "ok",
```

```
"timestamp": "2026-10-07T10:00:00" }
```

Evaluation

Import success rate	Fraction of required libraries that import without error	Target 100%
Smoke test	pytest runs an empty pipeline end to end on a dummy 5-second video	Pass/fail
Reproducibility	Run env_check twice and with the same seed; reports must match	Pass/fail

Unit tests

- Each import in a loop, collecting failures.
- ffmpeg subprocess return code is 0.
- Config loader rejects unknown keys.

Pass/fail gate

✓ All imports succeed; ffmpeg callable; CPU/GPU detected and logged.

✓ pytest passes; environment_report.json saved.

Connection to the next stages

The BaseStage contract (input artifacts in, one artifact out, metrics saved beside it) is the glue for the whole project. Each later phase simply declares which earlier artifacts it reads.

Common failures

- dlib is hard to install on Windows; prefer InsightFace/MediaPipe for faces.

- pyannote needs a Hugging Face token and accepted model terms.
- onnxruntime, torch and CUDA versions often conflict; use one environment per heavy component if needed, and communicate through files only

Phase 1: Video Ingestion and Validation

Prove the system can reliably accept a video, describe it, and produce the clean audio track that later phases need. This phase also fixes the time convention for the whole project.

Input	Video file or URL.
Tech stack	ffprobe/FFmpeg (ffmpeg-python), OpenCV VideoCapture, yt-dlp (URLs), hashlib (SHA-256).
Output artifacts	data/raw/{id}.mp4, data/metadata/{id}.json, data/audio/{id}.wav (16 kHz mono).
Feeds into	Phase 2, 3, 4, 9 (audio), all phases (video_id, fps).

Processing steps

1. If input is a URL, download with yt-dlp; otherwise copy to data/raw/ and compute SHA-256 as video_id.
2. Run ffprobe to read duration, fps, resolution, codec, rotation tag, audio stream.
3. Decode-test the file: `ffmpeg -v error -i f.mp4 -f null -`; any error output means corruption.

4. If the video has variable frame rate, re-encode to constant frame rate so that time = frame_index / fps holds.
5. Extract audio: `ffmpeg -i f.mp4 -ac 1 -ar 16000 f.wav` (Whisper and pyannote both expect 16 kHz mono).
6. Save metadata JSON.

Output format (example)

```
{ "video_id": "clip_001", "duration_s": 125.4, "fps": 29.97,
 "width": 1920, "height": 1080, "n_frames": 3759,
 "has_audio": true, "audio_path": "data/audio/clip_001.wav",
 "vfr": false, "sha256": "..." }
```

Evaluation

Metadata agreement	OpenCV frame count vs ffprobe frame count; relative difference	Target <= 1%
Audio/video length	|audio duration - video duration|	Target <= 0.1 s
Decode errors	Number of lines printed by the ffmpeg null decode	Target 0
Random frame check	Seek to 20 random timestamps, confirm frames are not black/corrupt	Pass/fail

Unit tests

- A truncated file is rejected with a clear error.
- A video without audio sets has_audio=false and skips audio extraction.
- A rotated phone video is upright after loading.

Pass/fail gate

✓ Video opens, metadata is consistent, audio is extracted, decode errors = 0.

Connection to the next stages

Time convention (fixed here for the whole project): all timestamps are float seconds from the start of the CFR video; frame index = round(t * fps). Phase 2 reads the WAV; Phase 4 reads the video and fps.

Common failures

- Variable frame rate causes drift between subtitle time and frame time.
- Phone videos carry a rotation flag that OpenCV may ignore.
- Very long videos: set a maximum duration in config for development (e.g. 5 minutes).

Phase 2: Speech Recognition (Whisper)

Convert audio into timestamped, readable dialogue segments. Segment IDs created here are the primary key that every later modality hangs on.

Input	data/audio/{id}.wav
Tech stack	OpenAI Whisper or faster-whisper; stable-ts or WhisperX for word-level timestamps; Silero VAD (optional, to suppress hallucination in silence); jiwer (WER/CER); srt library.

Output artifacts	data/subtitles/{id}.json and .srt
Feeds into	Phase 3 (alignment), 4 (candidate windows), 6 (CLIP text), 9 (text emotion), 12 (bubble text).

Processing steps

1. Load a Whisper model (tiny/base/small/medium chosen by config).
2. Transcribe with word timestamps enabled; optionally pre-filter with VAD.
3. Drop or flag segments with high no_speech_prob or very low avg_logprob (music lyrics, hallucinations such as 'thanks for watching').
4. Post-process into balloon-sized segments: split at punctuation to a maximum of about 12 words or 4 seconds; merge fragments shorter than 0.5 s.
5. Assign segment_id in time order; save JSON (with words) and SRT.

Output format (example)

```
{ "video_id": "clip_001", "model": "small",
 "segments": [ { "segment_id": 1, "start": 12.42, "end": 15.73,
 "text": "Where are you going?",
 "words": [ {"w": "Where", "start": 12.42, "end": 12.71} ],
 "avg_logprob": -0.31, "no_speech_prob": 0.02 } ] }
```

Evaluation

WER	jiwer.wer(reference, hypothesis) after lowercasing and removing punctuation; 5-10 clips of 1-3 min with hand-corrected transcripts	Reference transcripts
CER	Same, at character level (useful for names and non-English)	Reference transcripts
Timestamp error	Mark true start time of 30 utterances; mean absolute error of segment start	Manual marks
Hallucination rate	Segments in non-speech regions / total segments	Manual listening
Real-time factor	Processing time / audio duration, per model size	Timer
Model-size study	Table of tiny/base/small/medium: WER vs RTF; pick the knee of the curve	Above

Unit tests

- Segments are sorted and non-overlapping (or overlap is explained).
- No segment has end <= start.
- SRT round-trips through the srt parser.

Pass/fail gate

✓ Suggested starting thresholds (tune after your first run): WER <= 15% on clean dialogue, start-time MAE <= 0.5 s, hallucination rate <= 5%.

✓ Text readable on manual inspection of 10 clips.

Connection to the next stages

segment_id + [start, end] is used as a time window everywhere: Phase 3 attaches a speaker, Phase 4 pulls

candidate frames from the window, Phase 6 embeds the text, Phase 9 classifies its emotion, Phase 12 draws it.

Common failures

- CLIP accepts only 77 tokens, so long segments must be split (Phase 6 depends on this).
- Song lyrics and background chatter produce junk segments.
- Overlapping speech is usually transcribed as one speaker or dropped.

Phase 3: Speaker Diarization

Answer 'who spoke when' and attach an anonymous speaker label to every Whisper segment. Real character identity comes later (Phase 7).

Input	data/audio/{id}.wav, transcript.json
Tech stack	pyannote.audio (speaker-diarization pipeline), pyannote.metrics (DER), Silero VAD (alternative), scipy (Hungarian assignment for evaluation).

Output artifacts	data/diarization/{id}.json (turns + segment_speakers) and .rttm
Feeds into	Phase 7 (speaker to face), Phase 9 (audio emotion per turn), Phase 12 (balloon grouping).

Processing steps

1. Run the diarization pipeline on the WAV (pass min/max speakers if known).
2. Collect speaker turns (start, end, speaker).
3. Smooth: merge turns of the same speaker separated by less than 0.3 s; drop turns shorter than 0.3 s.
4. Align with Whisper: for each text segment, pick the speaker with the largest time overlap; store overlap_ratio.
5. If overlap_ratio is below 0.5 or two speakers overlap strongly, label the segment 'mixed' or 'unknown' rather than guessing.

Output format (example)

```
{ "turns": [ {"start": 12.4, "end": 15.7, "speaker": "SPEAKER_01"} ],
 "segment_speakers": [ {"segment_id": 1, "speaker": "SPEAKER_01",
 "overlap_ratio": 0.93} ] }
```

Evaluation

DER	Diarization Error Rate = (missed speech + false alarm + speaker confusion) / total reference speech. Use pyannote.metrics with a 0.25 s collar; reference RTTM made by hand in Audacity/ELAN for about 10 minutes	Manual RTTM
Segment attribution accuracy	Of N Whisper segments, fraction whose assigned speaker matches the manual label (after optimal label mapping)	Manual labels
Unknown rate	Fraction of segments labelled unknown/mixed	Automatic
Speaker-count error	|predicted speakers - true speakers|	Manual

Unit tests

• Every segment gets exactly one of: a speaker, 'mixed', 'unknown'.

• Label permutation between runs does not change DER (mapping is optimal).

• Turns never have negative duration.

Pass/fail gate

✓ Suggested: segment attribution accuracy >= 80% and DER <= 25% (film audio with music is hard; document the value you achieve).

✓ Output has text + timestamp + speaker for every segment.

Connection to the next stages

Speaker labels here (SPEAKER_00...) are arbitrary names. Phase 7 builds the mapping SPEAKER_xx to CHAR_yy by combining these turns with face tracks and lip movement.

Common failures

- Background music and sound effects raise DER sharply.
- Speaker labels are not stable across different videos.
- Gated Hugging Face model: accept terms and set the token before the first run.

Phase 4: Shot Detection, Frame Sampling and

Quality Filtering

Build the visual candidate pool: a set of good-quality frames per shot and per dialogue window, with black bars removed and near-duplicates dropped.

Input	data/raw/{id}.mp4, transcript.json
Tech stack	PySceneDetect (ContentDetector/AdaptiveDetector), OpenCV, variance-of-Laplacian blur score, imagehash (pHash + Hamming distance, as in the original CineComic), NumPy.
Output artifacts	data/shots/{id}.json, data/frames/shot_xxx/*.jpg
Feeds into	Phase 5 (face detection), 6 (CLIP image embeddings), 8 (selection), 10 (cropping).

Processing steps

1. Compute the black-bar crop box once (rows/columns whose mean intensity stays below a threshold across sampled frames).
2. Detect shot boundaries with PySceneDetect.
3. Sample frames: a fixed rate (about 2 fps) plus first, middle and last frame of every shot.
4. For each dialogue segment, take its window [start - 0.5 s, end + 0.5 s] intersected with its shot and cap at 8-12 candidates.
5. Compute per-frame quality: sharpness (Laplacian variance), brightness/contrast, black-bar flag.
6. Remove frames that are blurry (below a fraction of the video median sharpness) and near-duplicates (pHash Hamming distance <= 5).
7. Save JPEGs and the JSON index; store the crop box so later stages can map back to original pixels.

Output format (example)

```
{ "crop_box": [0, 140, 1920, 940],
 "shots": [ { "shot_id": 2, "start": 12.0, "end": 28.0,
 "frames": [ { "frame_id": "s02_f003", "t": 14.5,
 "sharp": 182.4, "phash": "a1b2c3d4e5f60718" } ] } ],
 "segment_candidates": { "1": ["s02_f002", "s02_f003", "s02_f004"] } }
```

Evaluation

Shot boundary P/R/F1	Match detected boundaries to manual boundaries within +/- 0.5 s (about 2 frames is stricter); 2-3 clips labelled by hand	Manual boundaries
Candidate coverage	Percent of dialogue segments with at least 3 candidate frames	Automatic; target 100%
Black-bar crop accuracy	IoU of automatic crop box vs manual box	Manual box
Filter sanity	Percent of frames removed as blurry/duplicate; contact-sheet check of 20 random shots	Visual
Pool size	Frames per minute (guards against disk/compute explosion)	Automatic

Unit tests

- Every candidate lies inside its shot's time range.
- No candidate contains black bars after cropping.
- Same input and config give the same frame list (determinism).

Pass/fail gate

✓ Suggested: boundary F1 >= 0.85, coverage 100%, no black bars, and a human looking at the contact sheet agrees the frames represent the shot.

Connection to the next stages

Each frame carries (frame_id, t, shot_id, crop_box). Phase 5 uses these frames for faces; Phase 6 embeds them; Phase 10 later needs the crop box to convert coordinates. Phase 7 reads the original video at full frame rate inside dialogue windows (active speaker detection needs consecutive frames, not just candidates).

Common failures

- Fades and dissolves produce missed or doubled boundaries.
- Flashes and explosions trigger false cuts.
- Blurry-by-design shots (motion) may be the important ones: make the blur filter relative, not absolute.

Phase 5: Face Detection, Tracking and Character ID (ArcFace)

Know which faces are on screen, follow them through each shot, and decide which faces are the same character across the whole video.

Input	Candidate frames and video frames inside dialogue windows (denser sampling for tracking).
Tech stack	InsightFace (SCRFD/RetinaFace detector, ArcFace 512-d embeddings), MediaPipe FaceMesh or 5/68-point landmarks (mouth position), IoU tracker or ByteTrack/SORT, scikit-learn agglomerative clustering (cosine), py-motmetrics (optional).
Output artifacts	data/faces/{id}.json, data/embeddings/faces_{id}.npy, data/tracks/{id}.json
Feeds into	Phase 7 (ASD), 8 (face visibility), 9 (face emotion crops), 10 (ROI), 12 (bubble tail and collision avoidance).

Processing steps

1. Detect faces on frames at about 5 fps inside dialogue windows; discard faces smaller than about 40 px or with low confidence.
2. Track within each shot: link boxes by IoU (and embedding similarity) into track_ids. 3. Compute an

ArcFace embedding per face; summarise each track by the mean of its top-quality faces.

4. Cluster track embeddings across shots with agglomerative clustering (cosine distance threshold) to get character_ids.
5. Keep landmarks, especially the mouth centre, for balloon tails and lip-motion analysis.
6. Mark low-confidence identities as UNKNOWN instead of forcing a cluster.

Output format (example)

```
{ "tracks": [ { "track_id": 7, "shot_id": 2, "character_id": "CHAR_01",
 "id_conf": 0.91,
 "frames": [ { "t": 14.5, "box": [412, 130, 560, 300],
 "mouth": [488, 262], "emb_idx": 132 } ] } ] }
```

Evaluation

Detection precision/recall	IoU >= 0.5 against 100-200 hand-labelled faces (CVAT/LabelImg), reported for small/medium/large and frontal/profile	Labelled faces
Tracking quality	ID switches and track fragmentation per shot; IDF1 or MOTA with py-motmetrics if you label a short clip densely	Labelled clip
Identity clustering	Pairwise F1, B-cubed F1 or purity of clusters against manual character labels on about 100 faces	Labelled faces
Unknown rate	Fraction of faces left UNKNOWN	Automatic

Unit tests

• A track never spans a shot boundary.

• Two tracks visible at the same time never share a character_id.

• Cluster assignments are stable when embeddings are re-run.

Pass/fail gate

✓ Suggested: recall >= 90% on medium/large frontal faces, zero ID switches inside a shot on test clips, cluster purity >= 90%.

✓ If CHAR_01 randomly becomes CHAR_07 between shots, do not continue.

Connection to the next stages

This phase produces the objects that the next phases bind to: **track_id** (visual entity in a shot) and **character_id** (identity across shots). Phase 7 links SPEAKER_xx to these; Phase 8 scores how visible the speaker's face is; Phase 12 uses the mouth position for the balloon tail.

Common failures

- Profile, occluded and tiny faces are missed; report this honestly.
- Animated characters break face recognisers trained on photos.
- One person under different makeup/lighting may split into two clusters; tune the threshold on labelled data.
- Face recognition is privacy-sensitive: process locally and use consented or open-licence videos.

Phase 6: CLIP Multimodal Alignment (core MML stage)

Measure, with a shared embedding space, how well each candidate frame matches what is being said. This replaces the original DSN-only keyframe scoring with a semantic signal.

Input	Candidate frames, transcript segments.
Tech stack	open_clip or Hugging Face CLIP (ViT-B/32 for speed, ViT-L/14 for quality), PyTorch, NumPy, scikit-learn metrics.

Output artifacts	data/embeddings/clip_img_{id}.npy, clip_txt_{id}.npy, data/alignment/{id}.json
Feeds into	Phase 8 (semantic score and diversity), Phase 13 (baseline comparison).

Processing steps

1. Encode all candidate frames with the CLIP image encoder (batch, cache to disk), L2-normalise.
2. Encode every segment with the CLIP **text** encoder (not a separate sentence encoder: both sides must live in the same space); truncate to 77 tokens.
3. Optionally encode a context string (previous + current + next segment) because single lines are often non-visual.
4. Compute cosine similarity matrix S[segment, frame] restricted to that segment's candidate frames. 5. Normalise within each segment (z-score or rank) so scores are comparable across segments. 6. Flag segments whose best similarity is low as 'non-visual dialogue' so Phase 8 can lean on other signals.

Output format (example)

```
{ "segment_id": 1, "candidates": [
 {"frame_id": "s02_f003", "clip_sim": 0.31, "z": 1.4, "rank": 1},
 {"frame_id": "s02_f002", "clip_sim": 0.24, "z": -0.2, "rank": 2} ],
 "non_visual": false }
```

Evaluation

Top-1 accuracy / Recall@K	A human marks the best (or all relevant) candidate frames for 50-100 segments; check whether CLIP's rank-1 / top-K hits them	Human relevance labels
MRR	Mean of 1/rank of the first relevant frame	Human labels
NDCG@K	Use graded labels (0 irrelevant, 1 ok, 2 best)	Graded labels
Baselines	Random frame, middle-of-segment frame, DSN score, CLIP. CLIP must beat random and middle-frame clearly	Same labels
Sanity control	Matched text-frame pairs should score above shuffled pairs in most cases	Automatic

Unit tests

- Embedding norms equal 1 after normalisation.
- Identical text gives identical embedding.
- A hand-picked obvious pair (e.g. 'a man driving a car' vs a car frame) ranks first.

Pass/fail gate

✓ CLIP selection beats random and middle-frame baselines on Recall@K/MRR by a clear margin, and failure cases (non-visual lines) are documented.

✓ If semantic matching does not work at all, stop and investigate before building on it.

Connection to the next stages

The similarity matrix S becomes $\mathbf{S_{semantic}}$ in Phase 8's scoring formula. The CLIP image embeddings are reused to compute frame-to-frame similarity for the diversity term, so no extra model is needed.

Common failures

- Real CLIP cosine similarities are small (roughly 0.15-0.35) and differences between frames are modest, so use within-segment normalisation; do not expect values like 0.91.
- CLIP is weak on counting, spatial relations and abstract dialogue.
- Movie stills differ from CLIP's web-image training data; fine-tuning is an optional extension, not a requirement.

Phase 7: Active Speaker Detection and Speaker-Face Matching

Decide which visible face is speaking, and build the mapping SPEAKER_xx to CHAR_yy so each dialogue line can be attached to a character (or marked off-screen/unknown).

Input	Diarization turns, face tracks, full-rate video frames and audio inside dialogue windows.
Tech stack	TalkNet-ASD or Light-ASD (pretrained on AVA-ActiveSpeaker); fallback baseline: mouth-aspect-ratio from landmarks correlated with VAD; scipy linear_sum_assignment (Hungarian algorithm).
Output artifacts	data/speaker/{id}.json
Feeds into	Phase 8 (speaker visibility), 9 (which face to read emotion from), 12 (bubble tail and attribution).

Processing steps

1. For each face track overlapping a speech segment, run ASD to get a speaking score over time (or compute the lip-motion baseline).
2. Local decision: for each segment, find the track with the highest mean speaking score inside the segment window.
3. Global mapping: build matrix M[speaker, character] = sum of overlap-weighted speaking scores over all segments; solve with the Hungarian algorithm for a one-to-one mapping.
4. Combine local and global evidence into a confidence (for example the margin between best and second-best).
5. If confidence is below threshold or no face is speaking, output on_screen = false / UNKNOWN. A wrong attribution is worse than none.

Output format (example)

```
{ "segment_id": 1, "speaker": "SPEAKER_01", "character_id": "CHAR_03",
 "track_id": 7, "conf": 0.84, "on_screen": true,
 "method": "asd+hungarian" }
```

Evaluation

Active speaker accuracy	Label 100 speaking events with the true speaking face; accuracy of the chosen track (mAP at frame level if using AVA-style labels)	Manual labels
Speaker-to-character accuracy	Fraction of segments where predicted character equals the true speaker	Manual labels
Selective accuracy	Plot accuracy vs coverage as the confidence threshold varies; report accuracy at a stated coverage	Same labels
Off-screen detection	Precision/recall of the on_screen=false decision	Manual labels
Baseline comparison	Original CineComic (no attribution), lip-motion only, ASD only, ASD + global mapping	Same labels

Unit tests

- Each speaker maps to at most one character.
- Segments with no visible face are never assigned a face.

• Confidence is in [0,1].

Pass/fail gate

✓ Suggested: accuracy >= 80% on assigned events at a documented coverage, with the abstain (UNKNOWN) option working.

✓ Below this, balloons must fall back to unattributed captions.

Connection to the next stages

Output attaches {speaker, character, track, confidence, on_screen} to every segment. Phase 8 rewards frames where that speaker's face is visible; Phase 9 reads facial emotion from that track; Phase 12 points the balloon tail at that face's mouth, or at the panel edge if off-screen.

Common failures

• Reaction shots: the listener is on screen while the speaker is off-screen, so a visible face does not imply a speaking face.

• Narration, voice-over and dubbed audio.

• Overlapping speech; several faces moving their lips.

Phase 8: Multimodal Keyframe Selection

Fuse all earlier signals into one score and pick the frames that become panels. This is the main research experiment of the project.

Input	Candidate frames, CLIP similarities, face/speaker data, quality scores, shot information.
Tech stack	NumPy, scikit-learn (grid search), CLIP embeddings (for diversity), optical flow or frame difference (motion), optional DSN score from the baseline.
Output artifacts	data/keyframes/{id}.json (ordered panels with score breakdown)
Feeds into	Phase 9, 10, 11, 12 and the baseline comparison in Phase 13.

Processing steps

1. Normalise each component to [0,1]: S_{sem} (CLIP), S_{spk} (speaker face visible, large, frontal), S_q (sharpness/exposure), S_{sal} (saliency/ROI prominence), S_{div} (1 - max cosine to already selected frames). 2. Score = $w_1S_{sem} + w_2S_{spk} + w_3S_q + w_4S_{sal} + w_5S_{div}$, with weights in configs/mml.yaml (never hard-coded). 3. Group consecutive segments in the same shot with the same speaker pair into one dialogue group; pick one frame (or two for long groups) per group using greedy selection with the diversity term (MMR style).

4. Visual-only branch for shots without dialogue: $w_1 = 0$, use saliency, quality, motion energy, shot duration and optional DSN score.

5. Apply a page budget (number of panels) by ranking groups on importance; then restore chronological order.

6. Tune weights by grid search on a validation clip set; report weights and the ablation of each term.

Output format (example)

```
{ "panels": [ { "order": 1, "frame_id": "s02_f003", "t": 14.5,
 "segment_ids": [1, 2], "character_id": "CHAR_03",
 "scores": {"sem": 0.8, "spk": 0.9, "q": 0.7, "sal": 0.6, "div": 1.0,
 "total": 0.82}, "branch": "dialogue" } ] }
```

Evaluation

Recall@K / precision	Humans pick 1-2 best frames per dialogue group; compare with the system's choice	Human selections
Method comparison	Uniform sampling, DSN (original), CLIP-only, full MML score	Same labels
Temporal coverage	Largest gap (seconds) between consecutive chosen frames; story beats missed	Automatic + manual
Redundancy	Fraction of panel pairs with CLIP cosine above 0.95 (near duplicates)	Automatic
Ablation	Remove one term at a time (no speaker, no quality, no diversity) and report the drop	Same labels
Preference test	Blind pairwise A/B: which panel set tells the story better	Participants

Unit tests

- Selected panels are in chronological order.
- No two panels are the same frame.
- Weights loaded from YAML change the output.

Pass/fail gate

✓ MML score is at least as good as the best baseline on Recall@K and wins the preference test, with no near-duplicate panels and no large story gaps.

Connection to the next stages

keyframes.json is the single hand-off to the comic-building phases. It keeps the score breakdown so you can explain every selected frame in the viva (explainability).

Common failures

- Weights tuned on test clips overstate results; use separate validation and test clips.
- Over-emphasising face visibility can pick listener reaction shots.
- Too many panels per page: enforce the budget.

Phase 9: Multimodal Emotion Fusion

Estimate the emotion of each dialogue line from text, voice and face, and fuse them robustly, including when a modality is missing.

Input	Transcript text, audio per speaker turn, speaker's face track crops.

Tech stack	Hugging Face Transformers (text emotion classifier, e.g. a RoBERTa-based 7-class model), wav2vec2/HuBERT speech-emotion model, face emotion model (e.g. HSEmotion/EmotiEffNet, DeepFace or FER+), PyTorch (optional fusion MLP/attention), scikit-learn metrics. Verify model availability and licences before choosing.
Output artifacts	data/emotions/{id}.json
Feeds into	Phase 12 (bubble shape and font style).

Processing steps

1. Fix one label set, the 7 MELD classes: anger, disgust, fear, joy, neutral, sadness, surprise. Write explicit mapping tables for models that use other labels (many speech models use only 4 IEMOCAP classes).
2. Text branch: classify the segment (optionally with previous line as context).
3. Audio branch: classify the speaker's audio for the segment window; mark as MASKED if the window is mostly non-speech or music.
4. Face branch: classify aligned crops of the speaker's face over all frames in the window and average; MASKED if the speaker is off-screen or the face is small/low confidence.
5. Fusion A (baseline): masked weighted late fusion of probability vectors, $p = \text{sum}(w_m * \text{mask}_m * p_m) / \text{sum}(w_m * \text{mask}_m)$.
6. Fusion B (advanced, optional): concatenation MLP or small attention layer trained on MELD with modality dropout so it learns to cope with missing inputs.
7. Map the fused label to a bubble style (Phase 12) and keep the confidence.

Output format (example)

```
{ "segment_id": 1,
 "text": {"label": "anger", "p": 0.82},
 "audio": {"label": "anger", "p": 0.89},
 "face": null,
 "fused": {"label": "anger", "conf": 0.86, "modalities": ["text", "audio"]} }
```

Evaluation

Dataset	MELD (7 emotions, text + audio + video, from the TV series Friends); also IEMOCAP if available. Use the official train/validation/test splits	Public dataset
Metrics	Accuracy, weighted F1 (the usual MELD metric), macro F1 (because 'neutral' dominates), per-class confusion matrix	Public labels
Modality ablation	Text, audio, face, T+A, T+F, A+F, T+A+F	Same data
Missing-modality test	Mask each modality at test time and report the degradation	Same data
Domain check	Label 50 segments from your own clips and compare	Manual labels
Calibration (optional)	Expected Calibration Error; temperature scaling per modality	Same data

Unit tests

- Fusion works when any one or two modalities are None.
- Output probabilities sum to 1.
- A MASKED modality never contributes weight.

Pass/fail gate

✓ Fused macro F1 is at least that of the best single modality and the pipeline never crashes with missing inputs.

✓ MELD is hard; report relative improvement and per-class behaviour rather than chasing absolute numbers.

Connection to the next stages

Only the final label and confidence flow forward. Phase 12 turns them into bubble shape (e.g. spiky for anger or surprise, wavy for fear, dashed for sadness, round for neutral/joy).

Common failures

- Sarcasm and context-dependent emotion.
- Class imbalance: accuracy alone is misleading.
- Acted TV dialogue differs from film; background music corrupts audio emotion.
- Face emotion models are weak on small, side-on or animated faces.

Phase 10: ROI Estimation and Comic Layout

Decide what must stay visible in each frame, choose a page template, and crop frames to panel shapes without cutting important content.

Input	keyframes.json, faces/tracks, frames, crop boxes.
Tech stack	OpenCV (spectral-residual saliency or the CAM approach from the original), optional U2-Net saliency, NumPy, template definitions (JSON), Pillow.
Output artifacts	data/layout/{id}.json (pages, panels, crop transforms)
Feeds into	Phase 11, Phase 12.

Processing steps

1. ROI per frame = union of face boxes (speaker's face weighted highest) + salient regions. 2. Choose a page template by number of panels and their importance (e.g. big panel for a key moment).
3. For each panel, slide a window of the panel's aspect ratio over the frame and choose the position that maximises ROI retention.
4. Reserve a bubble-safe area (low-ROI region) for text.
5. Store the crop transform (offset and scale) for every panel; Phase 12 needs it to convert face coordinates.

Output format (example)

```
{ "pages": [ { "page": 1, "template": "T3A", "panels": [
 { "panel_id": 1, "frame_id": "s02_f003", "crop": [120, 0, 1280, 720],
 "scale": 0.5, "roi_retention": 0.98, "safe_zones": [[0,0,400,120]] } ] } ] }
```

Evaluation

ROI retention	area(ROI inside crop) / area(ROI), mean and minimum across panels	Automatic
Face-cut rate	Fraction of panels where the speaker's face crosses the crop edge	Automatic; target 0
Aspect error	|crop aspect - panel aspect|	Automatic
Page fill	Percent of page area used; wasted space	Automatic
Visual check	Contact sheet of 20 pages	Manual

Unit tests

- Crops stay inside the frame.
- Crop aspect ratio equals panel aspect ratio.
- The same input gives the same layout.

Pass/fail gate

✓ Suggested: speaker face-cut rate = 0 and mean ROI retention >= 0.95.

Connection to the next stages

Coordinate chain (critical): original frame -> black-bar crop -> ROI crop -> resize to panel -> position on page. Every transform is stored so that a face or mouth point detected on the original frame can be mapped to the final page for the balloon tail.

Common failures

- Two far-apart faces cannot both fit a narrow panel: choose a wider template.

- Saliency may highlight bright backgrounds rather than characters.

Phase 11: Cartoonization

Give the selected panels a comic look. This runs after selection, so analysis always sees the original frames.

Input	Cropped panel images.
Tech stack	Classical: OpenCV bilateral filter + adaptive-threshold edges + k-means colour quantisation. Neural: AnimeGANv2 / CartoonGAN. Heavy option: diffusion img2img (slow, may change faces).
Output artifacts	data/outputs/panels_styled/*.png
Feeds into	Phase 12.

Processing steps

1. Apply the chosen style to the cropped panel at final resolution (not to the whole video).
2. Keep the original panel so that detections and coordinates still refer to un-stylised pixels.
3. Use one style and one parameter set for the whole comic for consistency.
4. Cache results per (frame, style, params).

Output format (example)

```
{ "panel_id": 1, "style": "opencv_bilateral_v1",
 "params": {"d": 9, "sigmaColor": 75, "k": 8}, "runtime_s": 0.21 }
```

Evaluation

Face preservation	Face detection rate on stylised vs original panels	Automatic; target >= 95% of original
Structure preservation	SSIM or edge-map overlap between original and stylised	Automatic
Style consistency	Colour-histogram or Gram-matrix distance across panels of one comic	Automatic
Runtime	Seconds per panel; seconds per minute of video	Timer
User rating	1-5 'looks like a comic' and 'still recognisable'	Participants

Unit tests

- Output size equals input size.
- Idempotent with a fixed seed/params.
- Faces still detected on a test set.

Pass/fail gate

✓ Faces remain detectable, style is consistent, runtime acceptable.

Connection to the next stages

The stylised image becomes the bottom layer of the panel; Phase 12 draws balloons on top using coordinates computed from the unstylised frame.

Common failures

- Neural styles can distort faces or erase fine detail.

• Edge-heavy filters create noisy panels on grainy video.

Phase 12: Speaker-Aware Bubbles, Page Assembly and Export

Combine text, speaker, emotion, ROI and layout into the final comic page, with balloons pointing at the right character and not covering important content.

Input	Layout, styled panels, transcript, speaker mapping, emotion labels, face/mouth coordinates.
Tech stack	Pillow (text fitting, drawing), SVG/HTML+CSS (as in the original output_template), reportlab or img2pdf (PDF), zipfile (CBZ).
Output artifacts	output/page.html, comic.pdf, comic.cbz, page.json
Feeds into	Phase 13 (final evaluation), the end user.

Processing steps

1. Map the speaker's mouth point through the stored transform chain to panel coordinates.
2. Generate candidate bubble positions around the panel; score each by overlap with faces/ROI, distance to the tail anchor, overlap with other bubbles, and reading order.
3. Fit text: wrap using font metrics, shrink font down to a minimum size; split long text across two bubbles.
4. Choose bubble shape from the fused emotion; draw the tail to the speaker's mouth; if off-screen, point to the panel edge on the speaker's side; if UNKNOWN, use a caption box.
5. Order bubbles by time within a panel (left-to-right, top-to-bottom).
6. Assemble pages; export HTML (baseline viewer style), PDF and CBZ.

Output format (example)

```
{ "page": 1, "panels": [ { "panel_id": 1, "bubbles": [
 { "segment_id": 1, "text": "Where are you going?", "shape": "spiky",
 "box": [40, 30, 280, 110], "tail": [300, 190], "speaker": "CHAR_03",
 "attributed": true } ] } ] }
```

Evaluation

Attribution accuracy	Tail points to the correct character; compare against manual labels on 100 bubbles	Manual labels
Bubble-face overlap	Fraction of bubbles covering any face (target 0)	Automatic
Bubble-ROI overlap	Average ROI area covered by bubbles	Automatic
Text overflow	Bubbles where text does not fit at minimum font size (target 0)	Automatic
Reading order	Pairs of bubbles in a panel in the wrong time order	Automatic
Readability	1-5 user rating; minimum font size	Participants

Unit tests

- Every bubble's text equals its segment text.
- Bubbles stay inside the panel.
- Exports open in a viewer (HTML, PDF, CBZ reader).

Pass/fail gate

✓ Zero overflow, zero face overlap, attribution accuracy not below Phase 7's accuracy.

Connection to the next stages

This phase consumes the outputs of Phases 2, 3, 7, 9, 10 and 11, which is why each of them stored IDs and coordinates in the same conventions.

Common failures

- Coordinate mapping errors (off by crop offset) place tails on the wrong person.
- Very long dialogue overwhelms small panels: condense or split.
- Unattributed speakers should be visibly neutral, not randomly assigned.

Phase 13: End-to-End Evaluation and Baseline Comparison

Turn the application into research: compare the original CineComic baseline with CineComic-MML on a fixed test set, with stage-level results, ablations and a user study.

Input	Baseline outputs (configs/baseline.yaml) and MML outputs (configs/mml.yaml) on the same clips.

Tech stack	pandas, matplotlib, scipy.stats (Wilcoxon signed-rank, bootstrap confidence intervals), Google Forms or similar for the study.
Output artifacts	results/*.csv, results/figures, final tables for the dissertation.
Feeds into	Report, viva and presentation.

Processing steps

1. Fix a test set of 8-10 clips (dialogue-heavy, multi-speaker, action, little dialogue) that were never used for tuning.
2. Run baseline and MML with identical seeds and settings except the configuration file. 3. Compute all

stage metrics (ASR, diarization, keyframes, speaker, emotion, bubbles) on both systems.

4. Run ablations: baseline, +CLIP, +speaker matching, +emotion fusion, full system, and full minus each component.
5. Run a user study: 15-20 participants, blind and randomised order, 5-point Likert ratings (readability, story faithfulness, speaker clarity, visual appeal) plus A/B preference.
6. Statistics: paired Wilcoxon signed-rank tests for ratings; bootstrap 95% confidence intervals for accuracy-type metrics.
7. Profile runtime and memory per minute of video; do a categorised failure analysis (what went wrong and why).

Output format (example)

```
system,keyframe_R@3,speaker_acc,emotion_macroF1,bubble_overlap,readability,sec_per_mi
n baseline,0.xx,n/a,0.xx,0.xx,3.x,xx
mml,0.xx,0.xx,0.xx,0.xx,4.x,xx
```

Evaluation

Component metrics	Table with baseline vs MML for each stage metric above	Earlier labels
Ablation table	Each added component and the gain it gives	Same data
User study	Mean +/- std and p-values per question; inter-rater agreement (e.g. Krippendorff alpha)	Participants
Efficiency	Seconds per minute of video, peak RAM/VRAM	Timer
Failure analysis	Categories with counts and example panels	Manual

Unit tests

• Both systems run from the same command with different configs.

• Results CSV regenerates from raw outputs with one script.

Pass/fail gate

✓ Every claim in the report is backed by a table row, with the baseline shown beside it and limitations stated.

Connection to the next stages

This is where all stage artifacts and logs are reused. Because each phase saved its metrics, the final tables are assembled, not re-invented.

Common failures

- Tuning on the test clips.
- Cherry-picked example panels: show failures too.
- Too few participants for statistical claims: state the sample size and limitations.

6. Datasets and Annotation Plan

Labelled data is what makes evaluation possible. Plan it early; it is the slowest part. Estimated hours are rough guides for one person.

ASR (Phase 2)	5-10 clips, 1-3 min each, hand-corrected transcript	Open-licence film clips, interviews, your own recorded dialogue	About 4 h per 10 min of audio
Diarization (3)	10 min with RTTM labels (who speaks when)	Audacity labels or ELAN	About 3 h
Shots (4)	2-3 clips with manual boundaries	Manual list of cut times	About 1 h
Faces/identity (5)	100-200 faces with box and character name	CVAT or LabelImg	About 3 h
Alignment (6)	50-100 segments with relevant frames ranked	Simple HTML/Excel sheet showing 8 candidates	About 3 h
Active speaker (7)	100 speaking events with true speaking face	Spreadsheet of (time, face)	About 2 h
Keyframes (8)	Best 1-2 frames per dialogue group	Same sheet as Phase 6	About 2 h
Emotion (9)	MELD train/val/test (public); 50 own segments	Download dataset; own labels in a sheet	About 1 h for own labels
User study (13)	15-20 participants	Google Forms with the comics as PDFs	About 2 days to collect

Choosing test videos

- Use videos you are allowed to process for research. Openly licensed films are the safest choice; for example, the Blender Foundation's open movies (Sintel, Big Buck Bunny, Tears of Steel) are released under Creative Commons licences. Tears of Steel is live action, which suits face and speaker stages. Check the current licence terms before use.
- Include variety: a dialogue-heavy two-person scene, a multi-speaker scene, a low-dialogue action scene, and a noisy-audio scene.
- For emotion, rely on MELD for the main numbers and use your own labelled segments only as a domain sanity check.
- Faces are personal data: process locally, avoid redistributing frames, and mention ethics and consent in your report.

7. Gate Summary Table

One-page checklist. Thresholds are suggested starting values (see the note on the cover). Adjust them after measuring the baseline, and write down why.

0 Environment	Imports, ffmpeg, GPU detection	All pass
1 Ingestion	Frame-count agreement, audio length, decode errors	<= 1%, <= 0.1 s, 0
2 Whisper	WER, start-time MAE, hallucination rate	<= 15%, <= 0.5 s, <= 5%
3 Diarization	DER, segment attribution accuracy	<= 25%, >= 80%
4 Shots/frames	Boundary F1, coverage	F1 >= 0.85, 100%
5 Faces/ID	Recall, ID switches, cluster purity	>= 90%, 0, >= 90%
6 CLIP	Recall@K/MRR vs random and middle-frame	Clearly better
7 Speaker-face	Accuracy at stated coverage	>= 80% at stated coverage
8 Keyframes	Recall@K vs baselines, duplicates	Best or equal, no duplicates
9 Emotion	Macro F1 fused vs best single	Fused >= best single
10 Layout	Speaker face-cut rate, ROI retention	0, >= 0.95
11 Cartoon	Face detection after styling	>= 95% of original
12 Bubbles	Overflow, face overlap, attribution	0, 0, >= Phase 7
13 End-to-end	Baseline vs MML with statistics	Evidence for each claim

Checkpoint rule

If a gate fails, stop and fix before going on. Typical stop conditions: timestamps wrong after Whisper; speaker turns unusable after diarization; frames not representing their shot; character IDs flipping between frames; CLIP matching no better than the middle frame; speaker-face matching poor. Moving ahead anyway makes every later number unreliable.

8. Baseline vs MML Configs and Ablation Design

8.1 Two configs, one command

```
# configs/baseline.yaml
keyframe_method: dsn
image_encoder: googlenet
speaker_matching: false
clip_alignment: false
emotion:
 text: true
 audio: false
 face: false
 fusion: false

# configs/mml.yaml
keyframe_method: multimodal
image_encoder: clip
text_encoder: clip
speaker_matching: true
active_speaker: true
emotion:
 text: true
 audio: true
 face: true
 fusion: late # or "learned"
weights: {sem: 0.35, spk: 0.25, q: 0.15, sal: 0.15, div: 0.10}
diversity: true
quality_filter: true

python main.py --config configs/baseline.yaml --video clip_001
python main.py --config configs/mml.yaml --video clip_001
```

The weights shown are placeholders; tune them on the validation clips.

8.2 Ablation table (fill this in during Phase 13)

A. Baseline (DSN)	no	no	text only		n/a	

B. + CLIP keyframes	yes	no	text only		n/a	
C. + speaker matching	yes	yes	text only			
D. + emotion fusion	yes	yes	T+A+F			
E. Full minus CLIP	no	yes	T+A+F			
F. Full minus speaker	yes	no	T+A+F		n/a	
G. Full minus face emotion	yes	yes	T+A			

Read this table as a story: each row adds or removes one component, and the change in the metric shows what that component is worth. Rows E-G show that each part of the system is necessary, not just decorative.

8.3 Expected results chart (illustrative layout only)

```
Keyframe relevance DSN |##############
 CLIP |################
 MML |##################
Speaker attribution Base |(none)
 MML |################
Emotion macro-F1 Text |#############
 T+A |###############
 T+A+F |#################
(Bars are a layout sketch, not predicted values. Replace with your own measurements.)
```


9. Metrics Cheat-Sheet

WER	(S + D + I) / N: substitutions, deletions, insertions over reference words	2
CER	Same at character level	2
DER	(missed speech + false alarm + speaker confusion) / total reference speech	3
Precision / Recall / F1	TP/(TP+FP), TP/(TP+FN), 2PR/(P+R)	4, 5, 7, 8
IoU	area(A and B) / area(A or B); a detection counts when IoU >= 0.5	4, 5, 10
IDF1 / ID switches	Tracking identity consistency; count of times a track changes identity	5
Cluster purity / B-cubed F1	How cleanly clusters correspond to true characters	5
Recall@K	Fraction of queries whose relevant item appears in the top K	6, 8
MRR	Mean of 1 / rank of first relevant item	6
NDCG@K	Ranking quality with graded relevance, discounted by rank	6
Selective accuracy	Accuracy on the subset where confidence exceeds a threshold, plotted against coverage	7
Accuracy / weighted F1 / macro F1	Weighted F1 weights classes by size (MELD standard); macro F1 averages classes equally	9
ECE	Expected calibration error: gap between confidence and accuracy	9

ROI retention	area(ROI inside crop) / area(ROI)	10
SSIM	Structural similarity between two images	11
Likert mean +/- std	5-point ratings from participants	13
Wilcoxon signed-rank	Paired non-parametric test for baseline vs MML ratings	13
Bootstrap CI	Resample items to get a 95% confidence interval for a metric	13
RTF	Processing time / media duration	2, 13

10. Risk Register and What to Cut

Model install conflicts (dlib, onnxruntime, CUDA)	High	Medium	Separate environments; prefer InsightFace/MediaPipe over dlib; communicate only through files
pyannote token or licence problems	Medium	Medium	Set up on day one; fallback: VAD-only segments plus clustering of speaker embeddings
Poor speaker attribution on film audio	High	High	Report selective accuracy; use UNKNOWN captions instead of guessing
CLIP weak on abstract dialogue	High	Medium	Context window of neighbouring lines; non-visual flag; rely on other score terms
Not enough labelled data	Medium	High	Start labelling in week 1; keep sets small but real; use MELD for emotion
Compute too slow	Medium	Medium	Cache everything; smaller models; shorter clips; sample fewer frames
Too many models for the time	High	High	Cut in the order below
User study participants	Medium	Medium	Recruit early; 10-15 is acceptable if stated honestly

Scope-cutting order if time runs short

1. Keep the core: Phases 0-4, 6, 8 (CLIP keyframes), 10-12 and the evaluation in 13.
2. Keep speaker matching (Phases 3, 5, 7); it is the most visible improvement.
3. Simplify emotion: text + audio only, or text + face with simple late fusion, instead of a learned fusion model.
4. Use classical cartoonization instead of a neural style.
5. Drop PDF/CBZ export and keep the HTML output.
6. Never cut evaluation. A smaller system with numbers beats a bigger one without.

11. Sixteen-Week Timeline

1	0, 1	Environment, ingestion, labelling plan; run original CineComic as the baseline	Baseline outputs for all clips
2-3	2, 3	Whisper, diarization, WER/DER tables	Gates 2 and 3
4	4	Shots and candidate frames	Gate 4
5-6	5	Faces, tracking, character IDs	Gate 5
7	6	CLIP alignment and the baseline comparison for keyframes	Gate 6
8-9	7	Active speaker + speaker-face mapping	Gate 7
10	8	Multimodal keyframe selection and ablation	Gate 8
11	9	Emotion fusion on MELD and own clips	Gate 9
12	10, 11	ROI, layout, cartoonization	Gates 10, 11
13	12	Bubbles, page assembly, export	Gate 12
14	13	End-to-end run on test clips, user study	Results tables
15	13	Statistics, failure analysis, figures	Final numbers
16	-	Report, presentation, demo video, viva practice	Submission

Start annotation in week 1 and annotate in parallel with coding; do not leave it to the end.

12. Folder Structure and Code Skeletons

```
CineComic_MML/
 |-- data/ raw/ audio/ subtitles/ diarization/ shots/ frames/ faces/ tracks/ |
 embeddings/ alignment/ speaker/ keyframes/ emotions/ layout/ outputs/ |-- src/
 | |-- ingestion/ video_loader.py metadata.py
 | |-- audio/ extract_audio.py whisper_asr.py diarization.py audio_emotion.py | |--
 video/ shot_detection.py frame_extraction.py black_bar.py quality_filter.py | |--
 vision/ face_detection.py tracking.py arcface.py clip_image.py | |-- alignment/
 clip_text.py similarity.py
 | |-- speaker/ active_speaker.py speaker_face.py
 | |-- keyframes/ scoring.py diversity.py selector.py
 | |-- emotion/ text.py audio.py face.py fusion.py
 | |-- layout/ roi.py layout.py crop.py
 | |-- comic/ cartoonize.py bubbles.py page.py
 | `-- evaluation/ asr_metrics.py diarization_metrics.py keyframe_metrics.py |
 emotion_metrics.py bubble_metrics.py end_to_end.py |-- configs/ baseline.yaml
 mml.yaml
 |-- tests/ notebooks/ results/ logs/
 `-- main.py
```

12.1 BaseStage skeleton

```
import json, time, hashlib
from pathlib import Path

class BaseStage:
 name = "base"
 requires = [] # names of earlier stages

 def __init__(self, cfg, video_id):
```

```
 self.cfg, self.video_id = cfg, video_id
 self.out = Path(cfg["data_dir"]) / self.name / f"{video_id}.json"
self.metrics_path = self.out.with_suffix(".metrics.json")

 def cache_key(self):
 return hashlib.sha1(json.dumps(self.cfg, sort_keys=True).encode()).hexdigest()[:8]

 def run(self): raise NotImplementedError
 def evaluate(self, gt): raise NotImplementedError # returns dict of metrics  def
gate(self, metrics): raise NotImplementedError # returns (passed, reason)

 def execute(self, gt=None):
 t0 = time.time()
 self.run()
 m = self.evaluate(gt) if gt else {}
 ok, why = self.gate(m) if m else (True, "no ground truth")
 m.update(runtime_s=time.time() - t0, passed=ok, reason=why, cfg=self.cache_key())
self.metrics_path.write_text(json.dumps(m, indent=2))
 return ok
```

12.2 Keyframe scoring skeleton (Phase 8)

```
import numpy as np

def score(frame, w):
 return (w["sem"] * frame["sem"] + w["spk"] * frame["spk"] +
 w["q"] * frame["q"] + w["sal"] * frame["sal"])

def select(cands, emb, w, k):
 chosen = []
 while len(chosen) < k and cands:
 def total(c):
 div = 1.0 if not chosen else 1 - max(
 float(emb[c["id"]] @ emb[s["id"]]) for s in chosen)
 return score(c, w) + w["div"] * div
 best = max(cands, key=total)
 chosen.append(best); cands.remove(best)
 return sorted(chosen, key=lambda c: c["t"]) # restore story order
```

12.3 Late emotion fusion skeleton (Phase 9)

```
import numpy as np
```



```
LABELS = ["anger","disgust","fear","joy","neutral","sadness","surprise"]

def fuse(probs, weights):
 # probs: {"text": vec or None, "audio": vec or None, "face": vec or None}
num, den = np.zeros(len(LABELS)), 0.0
 for m, p in probs.items():
 if p is None: # MASKED modality contributes nothing
 continue
 num += weights[m] * np.asarray(p); den += weights[m]
 if den == 0:
 return None, 0.0, []
 out = num / den
  return LABELS[int(out.argmax())], float(out.max()), [m for m, p in probs.items() if p is not None]
```

13. Corrections to Earlier Drafts

While assembling this guide, a few inconsistencies in earlier notes were fixed. They matter if you quote the notes in your report.

The plan said 12 phases but numbered up to 14	This guide uses Phases 0-13 (14 phases) consistently

Example CLIP similarities such as 0.91 / 0.54 / 0.22	These were illustrative. Real CLIP cosine similarities are small (roughly 0.15-0.35) with narrow gaps, so scores must be normalised within each segment
Face/ROI analysis after cartoonization in some diagrams	Detect faces and ROI on original frames; stylise only the final panel crops; draw balloons last
Active speaker detection after a single keyframe	ASD needs consecutive full-rate frames inside the dialogue window, so it reads the video directly, not only the candidate frames
Text embeddings drawn separately from CLIP image embeddings	Both sides must come from the same CLIP model (text encoder and image encoder) to share one space
Numeric thresholds written as fixed requirements	They are suggested starting points; set your own after measuring the baseline and justify them

About the CineComic internals

Details of the original project's internal modules that were not directly readable (the backend/ source and the project report) were described in the earlier overview as inferred. Before you cite the original's methods in your report, read its backend source and PROJECT REPORT.pdf yourself.

14. Viva Preparation

Why is this multimodal and not just a pipeline of separate models?	Three modalities (audio, text, vision) are combined, and CLIP gives a true joint text-image space; emotion and speaker matching fuse audio, text and video evidence. Fusion is late (decision level) except in the CLIP alignment.
Why CLIP instead of the original GoogLeNet + DSN?	DSN scores visual importance only; CLIP links the frame to what is being said, so panels match the dialogue. Evaluated in Phase 6 and 8 against DSN.
How do you know which face is speaking?	Active speaker detection from lip motion and audio, combined with diarization turns and a Hungarian assignment; low-confidence cases become UNKNOWN.
What happens if the speaker is off-screen?	on_screen = false; the balloon tail points to the panel edge, or a caption is used.
How do you handle a missing modality in emotion fusion?	A mask removes that modality from the weighted average (or the learned model was trained with modality dropout); evaluated by masking each modality at test time.
How do you avoid overfitting your thresholds?	Separate development, validation and test clips; test clips used once in Phase 13.
What are the limitations?	Non-visual dialogue, overlapping speech, off-screen speakers, small or animated faces, acted-emotion datasets, small labelled sets. All reported in the failure analysis.
What is the contribution?	A stage-wise, evaluated, speaker-aware and semantically aligned comic generator, compared against the original CineComic baseline with ablations and a user study.